

R Programming Collection

A Comprehensive Guide with Algorithms and Implementations

Table of Contents

1. Number Operations

- Armstrong Number Verification
- Factorial Calculation
- Fibonacci Series
- Palindrome Check
- Prime Number Check
- Combination (nCr) Calculator

2. Statistical Analysis

- Covariance and Correlation Calculation
- Custom Correlation Implementation

3. Machine Learning

- Decision Tree Classification
 - K-Means Clustering
 - Linear Regression
 - Logistic Regression
-

Number Operations

Armstrong Number Verification

Description:

An Armstrong number is an n-digit number that is equal to the sum of the nth powers of its digits.

Algorithm:

1. Convert the number to a string to iterate through its digits
2. Calculate the number of digits (n)
3. Extract each digit and raise it to the power of n
4. Sum these values
5. Compare the sum with the original number

6. Return TRUE if they match, FALSE otherwise

R

 Copy

```
armstrong <- function(num){  
  digits <- as.numeric(strsplit(as.character(num), "")[[1]])  
  n <- length(digits)  
  sum <- sum(digits^n)  
  
  return(sum == num)  
}  
  
num <- as.integer(readline("Enter a number:"))  
if(armstrong(num)){  
  print("armstrong")  
}else{  
  print("not armstrong")  
}
```

Factorial Calculation

Description:

Calculates the factorial of a given number. The factorial of n ($n!$) is the product of all positive integers less than or equal to n .

Iterative Approach

Algorithm:

1. Initialize result to 1
2. For each number from the input to 1, multiply the result by that number
3. Return the result

```
# Read input from the user
x <- as.integer(readline("Enter the number: "))

# Function to calculate factorial
factorial <- function(x) {
  val <- 1
  for (i in x:1) {
    val <- val * i
  }
  return(val)
}

# Call the function and print the result
result <- factorial(x)
cat("Factorial of", x, "is:", result, "\n")
```

Recursive Approach

Algorithm:

1. If input is 0, return 1 (base case)
2. Otherwise, return the input multiplied by the factorial of (input - 1)

```
n <- as.integer(readline("Enter the number: "))

fact1 <- function(x) {
  if (x == 0) {
    return(1)
  } else {
    return(x * fact1(x - 1))
  }
}

result <- fact1(n)
cat("The factorial of", n, "is", result, "\n")
```

Fibonacci Series

Description:

Generates the Fibonacci sequence up to the nth term. In this sequence, each number is the sum of the two preceding ones.

Algorithm:

1. Initialize the sequence with first two Fibonacci numbers [0, 1]
2. For each position from 3 to n:
 - Calculate the next number by adding the two previous numbers
 - Append to the sequence
3. Return the complete sequence

R

 Copy

```
fib <- function(n){  
  fib_series<-c(0,1)  
  for(i in 3:n){  
    next_term<- fib_series[i-1]+fib_series[i-2]  
    fib_series <- c(fib_series,next_term)  
  }  
  return(fib_series)  
}  
  
num<-as.integer(readline("Enter the num:"))  
result <- fib(num)  
cat(result)
```

Palindrome Check

Description:

Checks if a number reads the same backward as forward.

Algorithm:

1. Store the original number
2. Initialize reversed number to 0
3. While the number is greater than 0:
 - Get the last digit of the number using modulo 10
 - Add this digit to the reversed number (after multiplying the current reversed number by 10)
 - Remove the last digit from the original number by integer division by 10
4. Compare the reversed number with the original number
5. If they are equal, the number is a palindrome

```
n<- as.integer(readline("Enter the number:"))
rev <- 0
num <- n

while(n>0){
  a <- n %% 10
  rev <- (rev * 10) + a
  n <- n%% 10
}

if(rev==num){
  print(paste("Paliandrome"))
}else {
  print(paste("not paliandrome"))
}
```

Prime Number Check

Description:

Determines if a given number is prime. A prime number is a natural number greater than 1 that is not divisible by any number other than 1 and itself.

Algorithm:

1. If the number is less than or equal to 1, return FALSE
2. For each integer i from 2 to (number-1):
 - Check if the number is divisible by i (number % i == 0)
 - If divisible, return FALSE
3. If no divisors are found, return TRUE

```
n <- as.integer(readline("Enter the number:"))

prime <- function(x){
  if(x<=1){
    return(FALSE)
  }
  for(i in 2:(x-1)){
    if(x%%i==0){ # Note: This should check division by i, not 2
      return(FALSE)
    }
  }
  return(TRUE)
}

if(prime(n)){
  print("THE NUMBER IS PRIME")
}else{
  print("THE NUMBER IS NOT PRIME")
}
```

Combination (nCr) Calculator

Description:

Calculates the number of ways to choose r items from a set of n items, without regard to order.

Algorithm:

1. Define a factorial function:
 - Return 1 if input is 0
 - Otherwise calculate the product of numbers from 1 to input
2. Calculate nCr using the formula: $n! / (r! \times (n-r)!)$
3. Return the result

```
factorial <- function(x) {  
  if (x == 0) return(1)  
  return(prod(1:x))  
}  
  
nCr <- function(n, r) {  
  return(factorial(n) / (factorial(r) * factorial(n - r)))  
}  
  
n <- as.integer(readline("Enter value of n: "))  
r <- as.integer(readline("Enter value of r: "))  
result <- nCr(n, r)  
cat("The value of", n, "choose", r, "is:", result, "\n")
```

Statistical Analysis

Covariance and Correlation Calculation

Description:

Calculates the covariance and correlation coefficients between two variables in a dataset.

Algorithm:

1. Load a dataset (iris in this example)
2. Extract the relevant columns for analysis
3. Calculate the variance of a column
4. Calculate the covariance between two columns using built-in functions
5. Calculate the correlation between two columns using built-in functions
6. Display the results

```
# View the first few rows of the Iris dataset
data<-(iris)

# Extract the relevant columns
col1 <- iris$Sepal.Length
col2 <- iris$Sepal.Width
v<-var(col1)
print(v)

# Calculate Covariance
covariance <- cov(col1, col2)
cat("Covariance between Sepal.Length and Sepal.Width:", covariance, "\n")

# Calculate Correlation
correlation <- cor(col1, col2)
cat("Correlation between Sepal.Length and Sepal.Width:", correlation, "\n")
```

Custom Correlation Implementation

Description:

Manually calculates covariance and correlation between two variables without using built-in statistical functions.

Algorithm:

1. Load data from a CSV file
2. Calculate the mean of each variable
3. Calculate covariance using the formula: $\Sigma((x - \bar{x}) * (y - \bar{y})) / n$
4. Calculate standard deviations for both variables
5. Calculate correlation using the formula: $\text{covariance} / (\text{sd}_x * \text{sd}_y)$
6. Display the results


```
data <- read.csv("TomatoFirst.csv")

mean_x <- sum(data$Color) / length(data$Color)
mean_y <- sum(data$Texture) / length(data$Texture)

cov_xy <- sum((data$Color - mean_x) * (data$Texture - mean_y)) / length(data$Color)
cat("cov bt color and texture", cov_xy ,"\n")

sd_x <- sqrt(sum((data$Color - mean_x)^2) / length(data$Color))
sd_y <- sqrt(sum((data$Texture - mean_y)^2) / length(data$Texture))

cor_xy<- cov_xy / (sd_x * sd_y)
cat("cor bt color and texture",cor_xy,"\n")
```

Machine Learning

Decision Tree Classification

Description:

Implements a decision tree classification model on the Iris dataset with data splitting for training and testing.

Algorithm:

1. Load required libraries (caTools, rpart, rpart.plot)
2. Load the dataset (iris)
3. Set random seed for reproducibility
4. Split the data into training and testing sets (80% training, 20% testing)
5. Build a decision tree model on the training data
6. Make predictions on the test data
7. Visualize the decision tree
8. Calculate and print the accuracy
9. Generate and print the confusion matrix

```
library(caTools)
library(rpart)
library(rpart.plot)

data("iris")
set.seed(123)
split<-sample.split(iris$Species,SplitRatio = 0.8)
train_data<-subset(iris,split== TRUE)
test_data<-subset(iris,split== FALSE)

model<-rpart(Species ~.,data=train_data,method="class")

prediction<-predict(model,newdata=test_data,type="class")

rpart.plot(model)
accuracy=mean(prediction==test_data$Species)
cat("accuracy:",accuracy)
conf_matrix<-table(predcted=prediction,Actual=test_data$Species)
print(conf_matrix)
```

K-Means Clustering

Description:

Performs K-means clustering on the Iris dataset and visualizes the results.

Algorithm:

1. Load required libraries (ggplot2)
2. Load the dataset (iris)
3. Preprocess the data (remove NA values, select relevant features)
4. Set random seed for reproducibility
5. Apply K-means clustering with 3 centers
6. Assign cluster labels to the data
7. Extract cluster centroids
8. Plot the clustering results with labeled centroids

```
library(ggplot2)
df <- iris
print(colnames(df))
df <- na.omit(df[, 1:4])
set.seed(42)
model <- kmeans(df[, 1:4], centers = 3, nstart = 25)
df$cluster <- as.factor(model$cluster)
centroids <- data.frame(model$centers)
colnames(centroids) <- colnames(df[, 1:4])
print(centroids)
p <- ggplot(df, aes(x = Sepal.Length, y = Sepal.Width, color = cluster)) +
  geom_point(size = 2) +
  geom_point(data = centroids, aes(x = Sepal.Length, y = Sepal.Width),
            shape = 19, size = 4, color = "blue") +
  ggtitle("K-Means Clustering") +
  theme_minimal()
print(p)
```

Linear Regression

Description:

Implements a simple linear regression model to predict weight based on length.

Algorithm:

1. Load required libraries (caTools)
2. Create a sample dataset with length and weight values
3. Split the data into training and testing sets (70% training, 30% testing)
4. Build a linear regression model on the training data
5. Generate a model summary
6. Make predictions on the test data
7. Calculate the Mean Squared Error
8. Plot the regression line with the data points

```
library(caTools)

data <- data.frame(
  length = c(1, 2, 3, 4, 5, 6, 7),
  weight = c(6, 7, 8, 9, 12, 13, 14)
)

split <- sample.split(data$weight, SplitRatio = 0.7)
trainingset <- subset(data, split == TRUE)
testset <- subset(data, split == FALSE)

model <- lm(formula = weight ~ length, data = trainingset)

summary(model)

predicted_weight <- predict(model, newdata = testset)
print(predicted_weight)

mse <- mean(summary(model)$residuals^2)
cat("Mean Squared Error :", mse)

plot(data$length, data$weight, main = "Predicted Weight Based on Length",
      xlab = "Length", ylab = "Weight")
abline(model, col = "red", lwd = 2)
```

Logistic Regression

Description:

Implements a logistic regression model on the Titanic dataset to predict survival.

Algorithm:

1. Load required libraries (caTools)
2. Load and preprocess the Titanic dataset
 - Convert categorical variables to factors
 - Remove missing values
3. Split the data into training and testing sets (80% training, 20% testing)
4. Build a logistic regression model on the training data
5. Generate predictions on the test data
6. Convert probability predictions to binary class predictions (threshold = 0.5)

7. Calculate and print the accuracy
8. Plot the sigmoid curve with classified data points

R

 Copy

```
library(caTools)
titanic_data <- na.omit(read.csv("titanic.csv"))

titanic_data$Sex <- as.factor(titanic_data$Sex)
titanic_data$Embarked <- as.factor(titanic_data$Embarked)
titanic_data$Survived <- as.factor(titanic_data$Survived)

set.seed(123)
split <- sample.split(titanic_data$Survived, SplitRatio = 0.8)
train_data <- droplevels(subset(titanic_data, split == TRUE))
test_data <- subset(titanic_data, split == FALSE)

log_model <- glm(Survived ~ Pclass + Sex + Age + SibSp + Parch + Fare + Embarked,
                 data = train_data, family = binomial)

predictions <- predict(log_model, newdata = test_data, type = "response")
predicted_class <- ifelse(predictions > 0.5, 1, 0)

accuracy <- mean(predicted_class == as.numeric(as.character(test_data$Survived)))
cat("Accuracy:", accuracy, "\n")

x <- seq(-6, 6, 0.1)
plot(x, 1 / (1 + exp(-x)), type = "l", col = "black", lwd = 2,
     main = "Sigmoid Curve with Classified Data Points",
     xlab = "Logit", ylab = "Predicted Class (0 = Not Survived, 1 = Survived)")

predicted_logit <- log(predictions / (1 - predictions))
colors <- ifelse(predicted_class == 1, "red", "blue")
points(predicted_logit, predicted_class, col = colors, pch = 19)

legend("bottomright", legend = c("Survived", "Not Survived"), col = c("red", "blue"), pch = 19)
```